

Ticketless Parking

Dominik Schweigl, Nikola Adzic, Daniel Wenger

Universität Innsbruck

Motivation

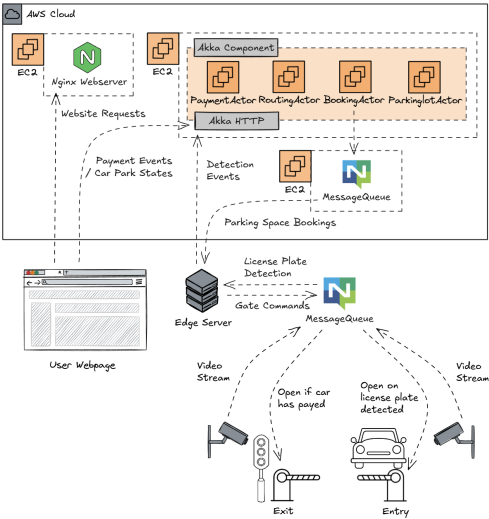
► Improved Parking Experience

- Pay online without physical tickets that need management and can be lost
- License plate-based identification
- No need to interact with payment terminals

► Smart Booking System

- Reserve parking spots in advance
- Find closest parking facilities with available spaces
- Extension: Book spots periodically (daily, weekly, monthly)

System Architecture



Resources (1/3)

► Dataset: Car License Plate Detection (Kaggle)

- 400+ annotated images (front/rear views)
- Suitable for prototyping and evaluation

► Computer Vision: YOLOv11+OCR

- Real-time license plate detection on the edge
- **OCR** extracts the license plate string

► Messaging: NATS

- Pub/Sub for camera frames and control signals (barrier)
- Asynchronous cloud-to-edge communication



↓ YOLOv11 License Plate Detection



↓ EasyOCR Text Extraction

MH15BD8877

Resources (1/3)

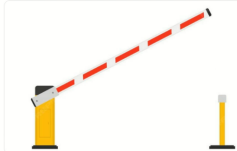
► Camera Implementation

- Python process simulating a camera at the entry/exit gate
- Sends image stream to edge server over NATS
- Car display duration 1 second
- Entry switches between car and empty gate scenes every 4 seconds
- Exit shows entered car exiting randomly

Barrier Dashboard

entry_0
ts: 2026-01-21T19:08:17.230939+00:00

Barrier

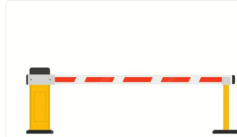


Camera



exit_0
ts: 2026-01-21T19:07:09.962030+00:00

Barrier



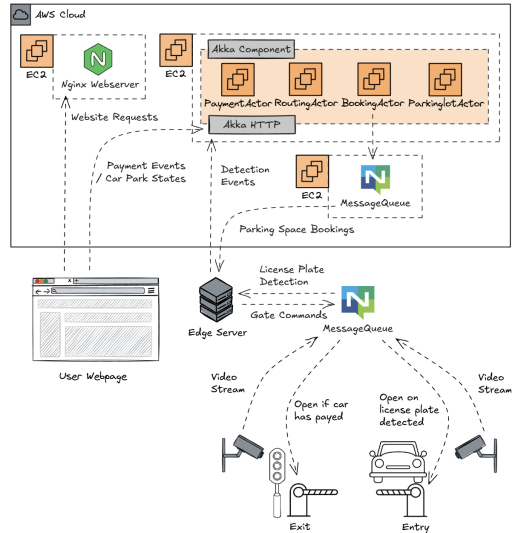
Camera



Resources (2/3)

► Edge Stack: Python + SQLite + FastAPI

- **Python** for ML integration
- **SQLite** as lightweight local store
- **FastAPI** to monitor barrier state and camera input
- **Dockerized** components for easy deployment on edge devices with docker-compose



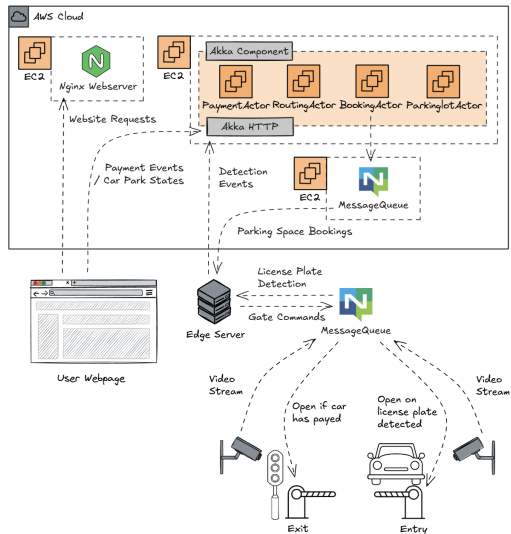
Resources (2/3)

► Edge Stack: Python + SQLite + FastAPI

- **Python** for ML integration
- **SQLite** as lightweight local store
- **FastAPI** to monitor barrier state and camera input
- **Dockerized** components for easy deployment on edge devices with docker-compose

► Cloud Stack: Akka Typed + Akka HTTP

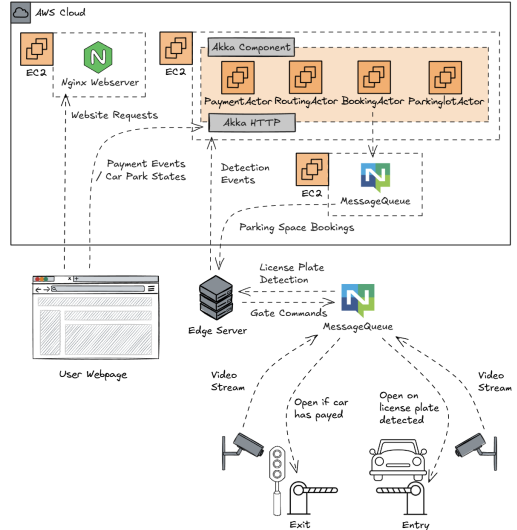
- Actor-based orchestration for *parking lot* functionality
- **REST API** for edge and web clients



Resources (3/3)

► Web Application: React + Tailwind CSS

- **React** provides a dynamic UI for availability, reservations, routing and payments
- **Tailwind** CSS for responsive layout
- Browser-based **geolocation** for nearby parking facilities



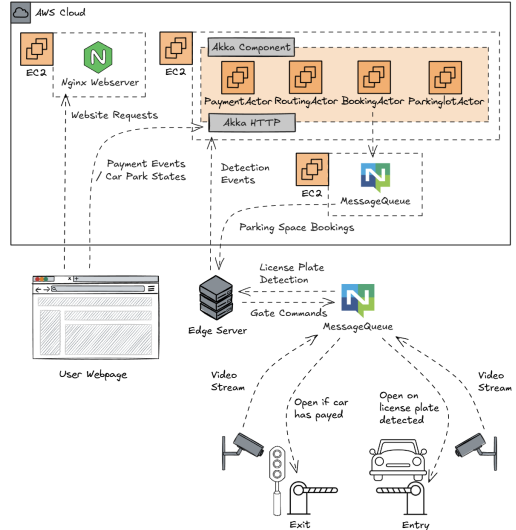
Resources (3/3)

► Web Application: React + Tailwind CSS

- **React** provides a dynamic UI for availability, reservations, routing and payments
- **Tailwind** CSS for responsive layout
- Browser-based **geolocation** for nearby parking facilities

► Hosting & Deployment: EC2 + Nginx + Terraform

- Separate EC2 instances for Akka backend, web frontend and cloud NATS
- **Nginx** webserver for web app
- **Terraform** for infrastructure as code; build artifacts uploaded via file provisioner



Live Demo

Video

Evaluation Setup

► Cloud Setup

- AWS EC2 t3.micro instances (2 vCPU, 1 GiB RAM)
- Separate instances for:
 - NATS server
 - Akka backend
 - Nginx webserver

Evaluation Setup

► Cloud Setup

- AWS EC2 t3.micro instances (2 vCPU, 1 GiB RAM)
- Separate instances for:
 - NATS server
 - Akka backend
 - Nginx webserver

► Edge Client simulation for load testing

- wrk2 HTTP benchmarking tool simulating client requests to backend
- Client simulation host with Intel i5-8600K CPU, 16 GiB RAM

Evaluation Setup

► Cloud Setup

- AWS EC2 t3.micro instances (2 vCPU, 1 GiB RAM)
- Separate instances for:
 - NATS server
 - Akka backend
 - Nginx webserver

► Edge Client simulation for load testing

- wrk2 HTTP benchmarking tool simulating client requests to backend
- Client simulation host with Intel i5-8600K CPU, 16 GiB RAM

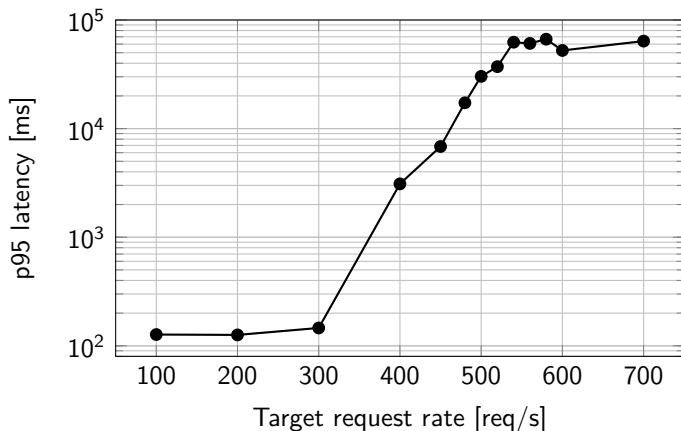
► Edge setup for latency measurements

- Host with Intel i5-8250U CPU, 8 GiB RAM, no GPU

Evaluation Result (1/3) Latency

Cloud Backend (Load Test)

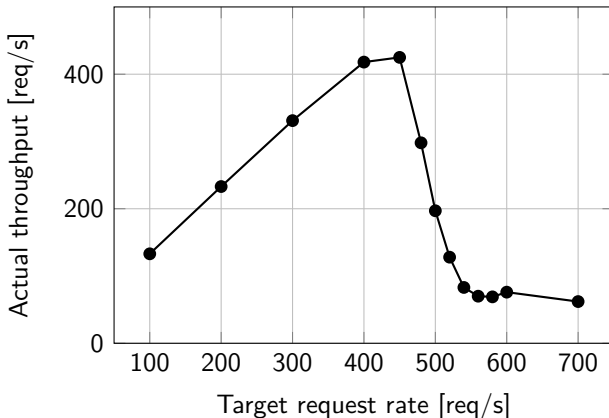
- ▶ Stable up to ≈ 300 req/s
- ▶ Saturation at ≈ 400 req/s
- ▶ Overload causes:
 - high tail latency
 - request timeouts



Evaluation Result (1/3) Throughput

Cloud Backend (Load Test)

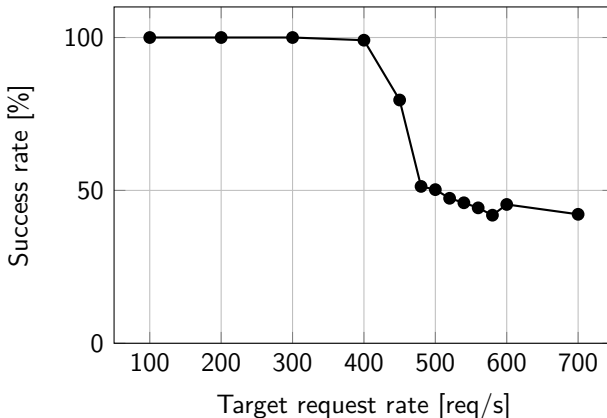
- ▶ Stable up to ≈ 300 req/s
- ▶ Saturation at ≈ 400 req/s
- ▶ Overload causes:
 - high tail latency
 - request timeouts



Evaluation Result (1/3) Success Rate

Cloud Backend (Load Test)

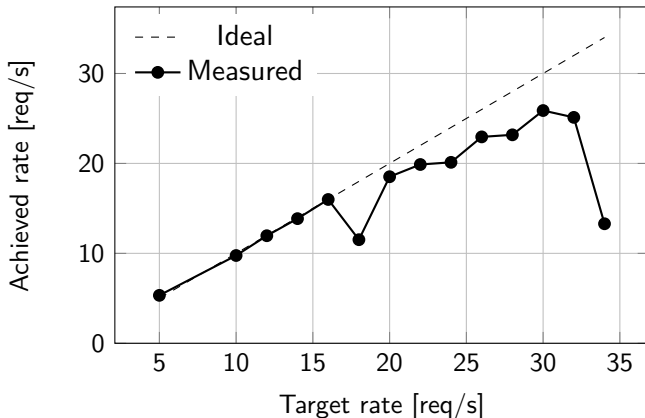
- ▶ Stable up to ≈ 300 req/s
- ▶ Saturation at ≈ 400 req/s
- ▶ Overload causes:
 - high tail latency
 - request timeouts



Evaluation Result (2/3) Single Client Nginx Throughput

Single Client Nginx Webserver (Load Test)

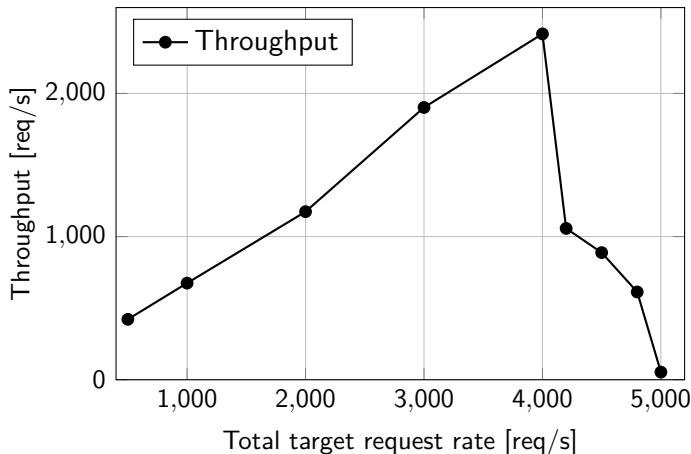
- ▶ For a single client request rates drop after **30 req/s**
- ▶ Limited inflight requests by wrk2 tool per client



Evaluation Result (2/3) Multi-Client Nginx Server Throughput

Multi-Client Nginx (Load Test)

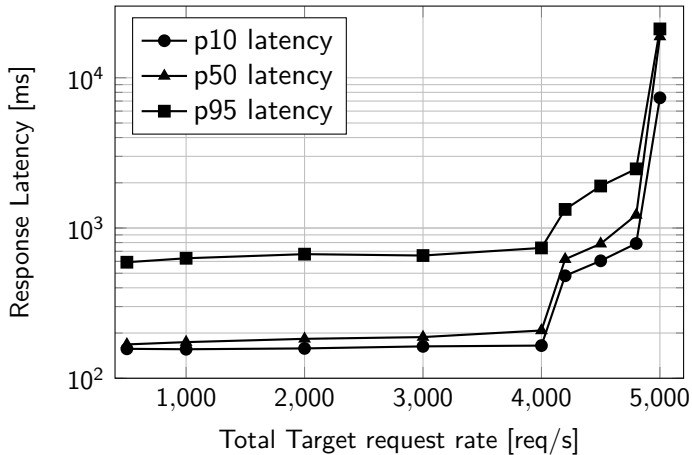
- ▶ Number of Clients scales with target request rate (5 req/s per client)
- ▶ Throughput grows linearly up to **4000 req/s**
- ▶ After **4000 req/s** throughput drops significantly



Evaluation Result (2/3) Multi-Client Nginx Server Response Latency

Multi-Client Nginx (Load Test)

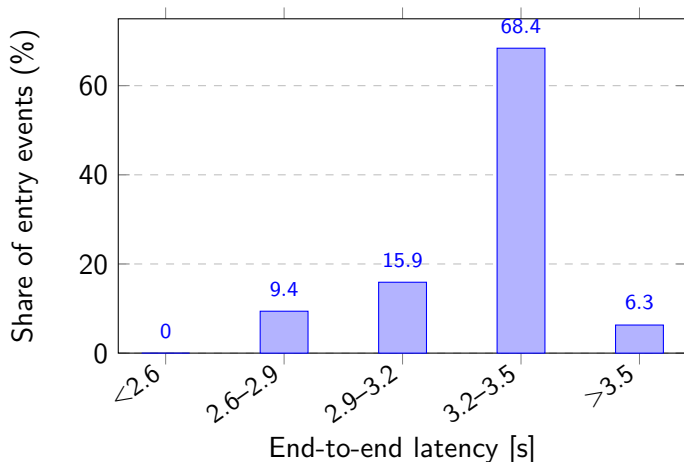
- ▶ Number of Clients scales with target request rate (5 req/s per client)
- ▶ Latency stable up to **4000 req/s**
- ▶ After **4000 req/s** latency increases significantly in accordance with throughput drop



Evaluation Result (3/3) Car Park Entry Gate Latency

Entry Gate Latency Distribution

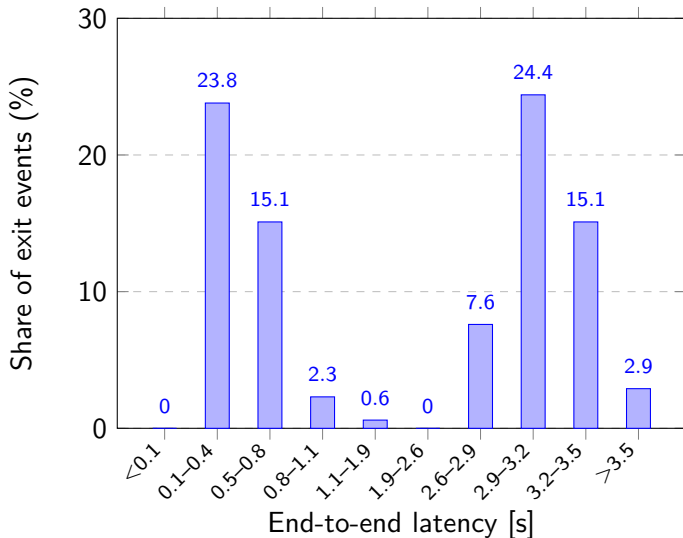
- ▶ End-to-end latency from image capture to barrier opening
- ▶ Includes:
 - Event transmission to cloud
 - License plate recognition
 - Database lookup
 - Barrier open command transmission
- ▶ Total entry samples: 414



Evaluation Result (3/3) Car Park Exit Gate Latency

Exit Gate Latency Distribution

- ▶ End-to-end latency from image capture to barrier opening
- ▶ Two scenarios:
 - Car with payment (high latency)
 - Car without payment (low latency)
- ▶ Total exit samples: 172



Thank you for your attention!

Questions?